A Closer Look at Methods and Classes (Chapter 7 of Schilit)

Object Oriented Programming BS AI Section A , B, D

By Rashida

Static keyword

Static Keyword

- a non-access modifier used to declare members (variables, methods, blocks, and nested classes) that belong to the class itself, rather than to any specific instance (object) of the class.
- This means that a single copy of a static member is shared by all objects of that class
- It can be accessed without creating an object.

The static keyword can be applied to:

- **Variables (Class Variables):** Shared among all objects and initialized once, making them ideal for constants or shared counters.
- **Methods (Class Methods):** Belong to the class, allowing access without an instance, often used for utility functions. These can only directly access other static members and cannot use this or super.
- **Blocks:** Used to initialize static variables with complex logic, running only once upon class loading.
- **Nested Classes:** Can exist without an instance of the outer class

Understanding *static* member variables

- static member variable of class
  - Associated with class
  - Independent of object
  - One copy for all objects
  - Accessed without any object reference (Main Method)

Understanding *static* member variables

- Instance variables which are static: Global Variables:
  - One copy is shared among all objects
- Change the value of static variable of one object:
  - Change will be visible to all objects, because there is only one copy for all of them

Static keyword

- Memory is allocated only once when the class is loaded.
- No object creation is needed to access static members; use the class name directly.
- Static methods and variables can’t access non-static members directly.
- Static methods can’t be overridden because they belong to the class, not instances

Static Keyword

- Variables
- Methods
- Blocks
- Nested classes

Static Variable

- A static variable is also known as a class variable. It is shared among all instances of the class and is used to store data that should be common for all objects.

Static Variable

- Demo

Static Blocks

- A static block is executed only once when the class is first loaded into memory.
- A static block in Java is used for static initialization of a class and its static variables.
- It is executed automatically by the JVM **only once** when the class is first loaded into memory, which occurs even before the main() method or any objects are created

- You can run your Java code written inside a static block without creating the main() method on JDK version 1.6 or previous.

Demo of static block

StaticDemo.java | StaticBlockDemo.java

```
1  class Demo{
2     static int val1;
3     int val2;
4     static{
5        val1 = 67;
6        System.out.println("The static block has been called.");
7     }
8  }
9  public class StaticBlockDemo{
10    public static void main(String args[]){
11       System.out.println(Demo.val1);
12    }
13 }
```

Static Methods

- A static method belongs to the class rather than to any object. It can be called directly using the class name.
- Can access only static data directly.
- Cannot access instance variables or methods directly.
- Cannot use this or super keywords.

Demo

StaticDemo.java | StaticBlockDemo.java | Calculator.java

```java
public class Calculator {
    // Static method for adding two numbers
    public static int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        // Calling the static method directly using the class name
        int result = Calculator.add(5, 3);
        System.out.println("Result: " + result);
    }
}
```

Static Nested Classes

- A static nested class is a class declared as static inside another class. It can be accessed without creating an object of the outer class

Tasks

- Can only call other static methods:
  - Let’s Try to call Non-Static method
- Can access only static member variables of class:
  - Let’s Try to access non-static members
- Can be called on class and the object reference:
  - Let’s try both
- Can’t refer *this* and *super* keywords
- Example

How to access them

- Outside the class in which they are defined:
  - Classname.methodname();
  - Classname.datamember;

Understanding *static* code block

- Gets executed exactly once at the start

- Example (static member variable, method and block)

Introducing keyword *final*

- Usage with variable:
  - To make them constant
  - Variables must be initialized if made final
    - Initialized when declared/Initialized with the help of constructor

```
final int FILE_NEW = 1;
final int FILE_OPEN = 2;
final int FILE_SAVE = 3;
final int FILE_SAVEAS = 4;
final int FILE_QUIT = 5;
```

Introducing keyword *final*

- Usage with Methods:
  - Method parameters can be made final
  - Prevents them from being changed inside method body
  - Local variables can also be final
  - Final can be used with methods to prevent overriding, which we will discuss latter

Nested and Inner Classes

- Nested class
  - Class within a class
  - Two types of nested class, *static* and *non-static*
  - Scope of nested is bound with outer class
  - If B is inside A, B will not exist until A is created (Dependency)
  - *non-static* are common
- Example

Nested and Inner Classes

- Static class:
  - Declared inside a class
  - Using static keyword
  - Can have static/non-static data members
  - Can have static/non-static methods
  - Can not access the non-static members of outer class directly, will have to use an object
    - Because of it they are rarely used

Demo

- Create a nested static class
- Add some static/non-static members and methods
- Try to access the outer class variables without creating object of outer class
- Try to create object in Main Method

Nested and Inner Classes

- Inner classes/ Non Static:
  - Nested/inner has access to member variables/methods including private of outer
  - Outer has no direct access of inner class methods/variables, an object of inner needs to be created

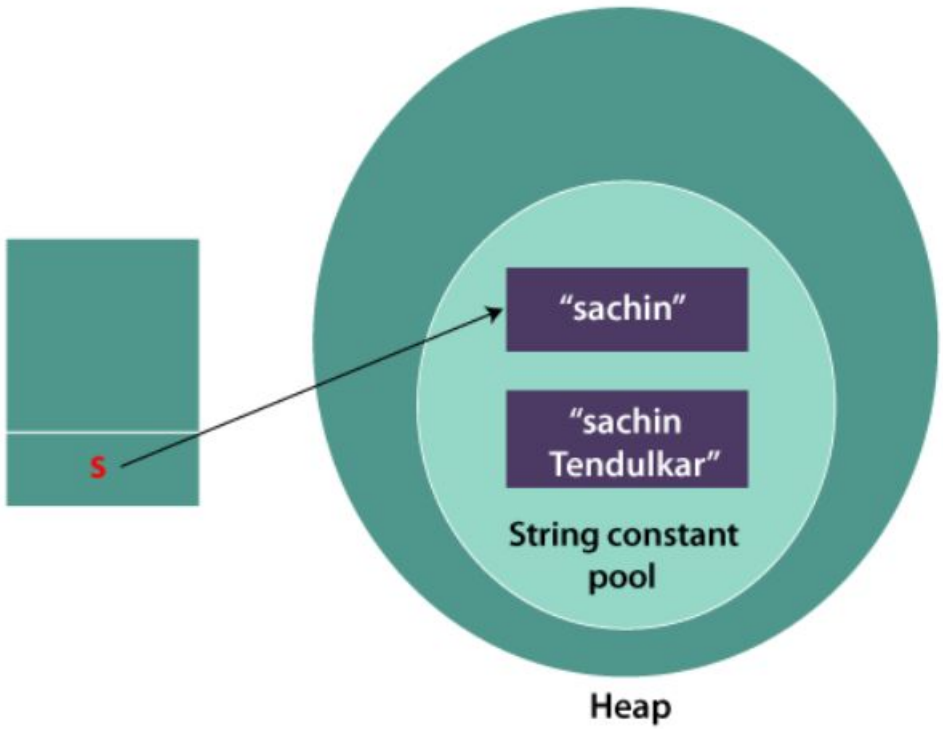
String class

- Class in java.lang
- Even string constants inside System.out.print() are string objects
- String objects are immutable
- StringBuffer and StringBuilder are mutatable strings

Strings are Immutable!

- Once you create a string, you can not change it’s value
- If you try to change the value a new string object is created
- But Why?
  - Strings are objects

```
String s="Sachin";
s.concat(" Tendulkar");//concat() method appends the string at the end
System.out.println(s);//will print Sachin because strings are immutable objects
```



Solution: Assign Reference to new string Object explicitly

```
public static void main(String args[]){
  String s="Sachin";
  s=s.concat(" Tendulkar");
  System.out.println(s);
 }
}
```

String class

- String methods
  - equals()
  - equalsIgnoreCase()
  - length()
  - charAt()

Demo

```
// Demonstrating some String methods.
class StringDemo2 {
  public static void main(String args[]) {
    String strOb1 = "First String";
    String strOb2 = "Second String";
    String strOb3 = strOb1;

    System.out.println("Length of strOb1: " +
                       strOb1.length());

    System.out.println("Char at index 3 in strOb1: " +
                       strOb1.charAt(3));

    if(strOb1.equals(strOb2))
      System.out.println("strOb1 == strOb2");
    else
      System.out.println("strOb1 != strOb2");

    if(strOb1.equals(strOb3))
      System.out.println("strOb1 == strOb3");
    else
      System.out.println("strOb1 != strOb3");
  }
}
```

String class

- equals() method vs == operator
  - equals() checks value
  - == checks reference

Varargs: Variable Length Arguments

- Support started with JDK 5
- If you are unsure about how many arguments a user will pass to method
- void add(int …arr):
  - Accepts zero or more arguments and places them in an array

Varargs: Variable Length Arguments

- We can have some mandatory arguments as well, so we need to place them before varargs
- Varargs is the last parameter of method

Varargs and Overloading

```java
public class varargsDemo
{
    public static void main(String[] args)
    {
        fun();
    }

    //varargs method with float datatype
    static void fun(float... x)
    {
        System.out.println("float varargs");
    }

    //varargs method with int datatype
    static void fun(int... x)
    {
        System.out.println("int varargs");
    }

    //varargs method with double datatype
    static void fun(double... x)
    {
        System.out.println("double varargs");
    }
}
```

Varargs and Method Overloading

```
// A method that takes varargs(here booleans).
static void fun(boolean ... a)
```

```
// A method takes string as a argument followed by varargs(here integers).
static void fun(String msg, int ... a)
```

Varargs and Ambiguity

```java
// A method that takes varargs(here integers).
static void fun(int ... a)

// A method that takes varargs(here booleans).
static void fun(boolean ... a)


// Calling overloaded fun() with different  parameter
fun(1, 2, 3); //OK
fun(true, false, false); //OK
fun(); // Error: Ambiguous!
```

Wrapper Classes

- provides the mechanism *to convert primitive into object and object into primitive*.
- AutoBoxing:
  - convert primitives into objects, automatically
- UnBoxing:
  - objects into primitives automatically

Wrapper Classes

Primitive Type	Wrapper class
boolean	Boolean
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double

Why need Wrapper Classes

- Primitive types are not objects
  - With wrapper classes java can be completely object oriented
- For working with collections we need them, as primitive types can not store null values

Boxing and AutoBoxing

```
//Java program to convert primitive into objects
//Autoboxing example of int to Integer
public class WrapperExample1{
public static void main(String args[]){
//Converting int into Integer
int a=20;
Integer i=Integer.valueOf(a);//converting int into Integer explicitly
Integer j=a;//autoboxing, now compiler will write Integer.valueOf(a) internally

System.out.println(a+" "+i+" "+j);
}}
```

Unboxing

```
//Java program to convert object into primitives
//Unboxing example of Integer to int
public class WrapperExample2{
public static void main(String args[]){
//Converting Integer to int
Integer a=new Integer(3);
int i=a.intValue();//converting Integer to int explicitly
int j=a;//unboxing, now compiler will write a.intValue() internally

System.out.println(a+" "+i+" "+j);
}}
```

Random

- A class in java.util
- Methods for generating random numbers
- nextInt()
- nextInt(int n): generates random number between 0 and n
- nextInt(-value): gives error

String vs StringBuffer vs StringBuilder

- String is immutable
- StringBuffer and StringBuilder are mutable
  - StringBuilder is faster
  - Both have a method called append

  - System.identityHashCode(s1)

Tasks

- Write a Java method to calculate the average of a variable number of doubles using varargs.
- Write a java program:
  - You will create a method which accepts variable number of integer arrays and print their elements